Table des matières

Cours 06 — Modéliser les autres faits 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Les vues des deux journaux · STAGING 2
Étape 2 — Le fait d'accumulation, `fct_lots` · MARTS 3
Étape 3 — Le fait sans mesure, `fct_consentements` · MARTS 5
Étape 4 — Lire les deux faits · MARTS 7
Étape 5 — Les tests des deux faits · MARTS 9
Étape 6 — Versionner · MARTS 11
À retenir 11

Cours 06 — Modéliser les autres faits

Cours 6/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
		✓ ici

Ce que vous allez faire

L'étoile des ventes enregistre des transactions : une vente arrive, sa ligne s'écrit, puis ne bouge plus. Le café a deux autres processus, que la vente ne montre pas. Un lot du torréfacteur avance par étapes, de la charge au premier craquement, puis à la sortie : il devient un fait d'accumulation, une ligne par lot et une colonne par étape. Un membre accorde, refuse ou retire son consentement à l'infolettre : il n'y a rien à additionner, et ce fait sans mesure se lit en comptant ses lignes. Le cours finit par leurs tests, dont un témoin : la dernière décision de chaque membre doit tomber sur sa fiche.

les deux journaux de la `RAW` → `stg_lots`, `stg_consentements` → `fct_lots`, puis `dim_date` étendue et `fct_consentements` → la lecture dans le notebook → les tests et `dbt build`

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_06_AE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, dbt et Git ; la seconde pour le notebook de DuckDB, qui s'ouvre à l'étape 4. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt build
REM Attendu : 26 of 26 : neuf modèles, le snapshot et 16 tests, puis Done. PASS=26 WARN=0 ERROR=0.
```

Ce cours part du projet tel que le cours 05 l'a laissé. S'il manque un test ou le snapshot, reprenez la fin du cours 05 avant de continuer.

Étape 1 — Les vues des deux journaux · STAGING

Deux tables de la `RAW` attendaient ce cours : `raw_lots`, le journal du torréfacteur, un lot par ligne, et `raw_consentements`, le journal des consentements, une décision d'un membre par ligne. Le dictionnaire des sources les décrit, sections 4 et 3. Chacune prend sa vue, par les gestes du cours 03 : se déclarer comme source, puis se faire typer. Chaque journal n'a qu'un export : rien à dédoublonner. Rien de personnel non plus : une décision ne porte que le numéro de la carte, que `dim_membre` garde déjà.

1.1 Déclarer les deux journaux, dans la première fenêtre.

```
notepad 02_ae\models\staging\_sources.yml
```

Ajouter ces deux entrées au bout de la liste des tables, alignées sur les cinq autres :

```
      - name: raw_lots
        description: "Le journal des lots du torréfacteur, un lot par ligne, en juin 2025."
      - name: raw_consentements
        description: "Le journal des consentements des membres, une ligne par décision, depuis 2023."
```

Enregistrer, puis fermer le Bloc-notes.

1.2 Écrire `stg_lots`.

```
notepad 02_ae\models\staging\stg_lots.sql
```

Le début de la charge devient un horodatage, les secondes des entiers. Les deux poids passent en décimaux au centième, comme les montants, et la température de sortie au dixième, `decimal(10, 1)`. Accepter la création, puis écrire :

```
-- 02_ae/models/staging/stg_lots.sql : les lots du torréfacteur, typés, un par lot
select
    -- 1. typer, dans l'ordre du dictionnaire
```


© 2026 iAlexMG. Tous droits réservés.

```
    lot_id,
    debut::____ as debut,
    produit_id::integer as produit_id,
    profil,
    poids_vert_kg::decimal(10, 2) as poids_vert_kg,
    -- puis, à écrire : poids_torrefie_kg, premier_craquement_s, duree_s, temp_sortie, statut et remarque
-- 2. lire la RAW par sa source ; le journal n'a qu'un export, rien à dédoublonner
from ____
```

Enregistrer, puis fermer le Bloc-notes.

1.3 Écrire `stg_consentements`.

```
notepad 02_ae\models\staging\stg_consentements.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/staging/stg_consentements.sql : le journal des consentements, typé, une ligne par décision
select
    -- 1. typer, dans l'ordre du dictionnaire
    membre_id,
    horodatage::____ as horodatage,
    -- puis, à écrire : consentement, statut et canal, tels quels
-- 2. lire la RAW par sa source ; le journal n'a qu'un export, rien à dédoublonner
from ____
```

Enregistrer, puis fermer le Bloc-notes.

1.4 Bâtir les deux vues.

```
dbt run --select stg_lots stg_consentements
REM Attendu : 2 of 2 OK created sql view model, main.stg_consentements et main.stg_lots, puis PASS=2.
```

Si dbt s'arrête sur	La cause, et le geste
`depends on a source named 'brulerie.raw_lots' which was not found`	le journal n'est pas déclaré : reprendre l'étape 1.1

Étape 2 — Le fait d'accumulation, `fct_lots` · MARTS

Un fait n'enregistre pas toujours une vente. On en distingue quatre familles, selon ce que le processus produit :

© 2026 iAlexMG. Tous droits réservés.

La famille	Ce qu'elle enregistre	Ce qui arrive à sa ligne	À La Brûlerie
Transaction	un événement, à un instant	rien : elle s'écrit une fois	`fct_ventes`, une ligne d'un ticket
Accumulation	un dossier qui avance par étapes	elle se remplit à chaque étape franchie	`fct_lots`, un lot du torréfacteur
Instantané périodique	un état, relevé à intervalle régulier	rien : une ligne par relevé	les sondes du torréfacteur, relevées aux 10 secondes, que la `RAW` d'AE ne porte pas
Sans mesure	un événement dont seule l'occurrence compte	rien : elle s'écrit une fois	`fct_consentements`, une décision d'un membre

Un lot est un dossier : chargé dans le tambour, il craque une première fois, puis il en sort. Le fait d'accumulation lui donne une ligne, et une colonne par jalon, une étape datée du dossier :

Le jalon	La colonne	Tirée du journal
la charge	`charge_le`	`debut`
le premier craquement	`craquement_le`	`debut`, plus `premier_craquement_s` secondes
la sortie	`sortie_le`	`debut`, plus `duree_s` secondes

Le journal donne les jalons en secondes depuis la charge ; le fait les rend en heures, qui se lisent seules. `premier_craquement_s * interval 1 second` fait d'un nombre de secondes une durée, qui s'ajoute à un horodatage. Les durées entre deux jalons se calculent une fois, dans le fait, plutôt que dans chaque rapport : jusqu'au craquement, puis du craquement à la sortie, ce que le torréfacteur appelle la phase de développement.

Le fait garde aussi les deux poids, le vert chargé et le torréfié pesé à la sortie : ils s'additionnent d'un lot à l'autre. La perte de poids, un rapport entre les deux, ne s'additionne pas : elle se calcule à la lecture, sur les sommes.

Ses clés suivent celles de `fct_ventes` : le jour de la charge, et le café torréfié, par le même `md5` que `dim_produit`. Le Mélange du Passage se torréfie au tambour et se vend en sac au comptoir : une seule dimension des produits sert les deux faits. On dit qu'elle est conforme, et c'est elle qui permet de mettre côte à côte le café torréfié et le café vendu.

2.1 Écrire le fait, dans la première fenêtre.

```
notepad 02_ae\models\marts\fct_lots.sql
```

Accepter la création, puis écrire :

© 2026 iAlexMG. Tous droits réservés.

```
-- 02_ae/models/marts/fct_lots.sql : le fait d'accumulation des lots, une ligne par lot, un jalon par colonne
select
    -- 1. une clé vers chaque dimension : le jour de la charge, le café torréfié
    debut::date as cle_date,
    ____ as cle_produit,
    -- 2. le lot, gardé dans le fait, avec son profil et son statut
    -- 3. les jalons : la charge, le premier craquement, la sortie
    debut as charge_le,
    debut + premier_craquement_s * interval 1 ____ as craquement_le,
    -- puis, à écrire : sortie_le, de la même façon, avec duree_s
    -- 4. les durées entre les jalons, en secondes, puis les deux poids
    premier_craquement_s as secondes_avant_craquement,
    ____ as secondes_de_developpement,
    -- puis, à écrire : poids_vert_kg et poids_torrefie_kg
from {{ ref('____') }}
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Bâtir le fait.

```
dbt run --select fct_lots
REM Attendu : 1 of 1 OK created sql table model main.fct_lots, puis Done. PASS=1.
```

Étape 3 — Le fait sans mesure, `fct_consentements` · MARTS

Une décision d'un membre sur l'infolettre ne porte ni quantité ni montant : un membre, un instant, ce qu'il a décidé, et où. C'est pourtant un fait : son occurrence est l'information, et le compte de ses lignes en est la mesure. On l'appelle un fait sans mesure. La fiche du membre, dans `stg_membres`, ne dit que l'état au 30 juin : il consent, ou non. Le journal dit comment on y est arrivé : qui a accepté à l'inscription, et qui s'est désabonné depuis. Le fait garde chaque décision, avec sa date et son membre comme clés, et ses codes tels quels, comme le ticket dans `fct_ventes`.

Reste la date. Le cours 04 a bâti `dim_date` pour les ventes : les 30 jours de juin. Le journal remonte au 1er mai 2023, l'ouverture du programme de fidélité, et 1 017 de ses 1 054 décisions tombent avant juin : leur clé ne trouverait aucun jour dans la dimension, et le test `relationships` de l'étape 5 les rendrait toutes en faute. Une dimension conforme couvre la période de chaque fait qui la lit : `dim_date` s'étend au 1er mai 2023. Elle reste fabriquée, sans source.

Les trois dimensions servent maintenant chacune deux faits ou plus. Le tableau qui croise les faits et les dimensions qu'ils partagent s'appelle la matrice de bus : il se lit avant d'ajouter un fait, pour savoir quelles dimensions il reprendra.

Le fait	**dim_date**	**dim_produit**	**dim_membre**
`fct_ventes`	le jour de la vente	le produit vendu	le membre, s'il présente sa carte

© 2026 iAlexMG. Tous droits réservés.

Le fait	dim_date	dim_produit	dim_membre
fct_lots	le jour de la charge	le café torréfié	
fct_consentements	le jour de la décision		le membre qui décide

3.1 Étendre la dimension des dates, dans la première fenêtre.

```
notepad 02_ae\models\marts\dim_date.sql
```

Trois lignes changent : la première, qui dit ce que fait le modèle ; le commentaire 4 ; la première date de la série. Le reste ne bouge pas :

```
-- 02_ae/models/marts/dim_date.sql : les jours du 1er mai 2023 au 30 juin 2025, un par ligne
select
    -- 1. la clé : la date elle-même
    jour::date as date_jour,
    -- 2. le nom du jour, du lundi (1) au dimanche (7)
    case isodow(jour)
        when 1 then 'lundi' when 2 then 'mardi' when 3 then 'mercredi'
        when 4 then 'jeudi' when 5 then 'vendredi' when 6 then 'samedi'
        when 7 then 'dimanche'
    end as nom_jour,
    -- 3. la fin de semaine : le samedi et le dimanche
    isodow(jour) >= 6 as est_fin_de_semaine
-- 4. une ligne par jour, de l'ouverture du programme de fidélité au 30 juin, les deux bornes comprises
from generate_series(date '____', date '2025-06-30', interval 1 day) as jours(jour)
```

Enregistrer, puis fermer le Bloc-notes.

3.2 Écrire le fait.

```
notepad 02_ae\models\marts\fct_consentements.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/models/marts/fct_consentements.sql : le fait sans mesure des consentements, une ligne par décision
select
    -- 1. une clé vers chaque dimension : le jour de la décision, le membre qui l'a prise
    horodatage::____ as cle_date,
    ____ as cle_membre,
    -- 2. la décision, gardée dans le fait : son instant, ce qu'elle vise, ce qu'elle dit, son canal
-- 3. aucune mesure : le compte des lignes est l'information
from {{ ref('stg_consentements') }}
```

Enregistrer, puis fermer le Bloc-notes.

© 2026 iAlexMG. Tous droits réservés.

3.3 Bâtir la dimension et le fait.

```
dbt run --select dim_date fct_consentements
REM Attendu : 2 of 2 OK created sql table model, main.dim_date puis main.fct_consentements, et PASS=2.
```

Étape 4 — Lire les deux faits · MARTS

4.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « AE Cours 06 », puis attacher l'entrepôt dans la première cellule :

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases, avec fct_lots et fct_consentements.
```

4.2 Lire les jalons du premier lot du mois, dans une cellule neuve.

```
select lot_id, charge_le, craquement_le, sortie_le, secondes_avant_craquement,
       secondes_de_developpement
from brulerie.fct_lots
where lot_id = 'L-2025-0214';
-- Attendu : chargé le 2025-06-03 à 07:00:00, premier craquement à 07:11:00, sortie à 07:13:00 ;
-- 660 secondes avant le craquement, 120 de développement.
```

4.3 Relire le même lot à 7 h 12. Le journal du torréfacteur s'exporte une fois le mois fini, et chaque lot y arrive complet. Branché sur le torréfacteur, le fait recevrait le lot dès la charge : sa ligne existerait déjà, ses jalons à venir vides, et chaque chargement suivant la remplirait. Le grain ne change pas, une ligne par lot, mais la ligne se met à jour, et un jalon vide est une information : le lot est encore dans le tambour. La requête relit le lot tel que le fait l'aurait porté à 7 h 12.

```
-- 1. le lot L-2025-0214, tel que le fait l'aurait porté à 7 h 12 : un jalon pas encore franchi reste vide
select lot_id, charge_le,
       case when craquement_le <= timestamp '2025-06-03 07:12:00' then craquement_le end as craquement_le,
       case when sortie_le <= timestamp '2025-06-03 07:12:00' then ____ end as sortie_le
from brulerie.fct_lots
where lot_id = 'L-2025-0214';
-- Attendu : chargé à 07:00:00, premier craquement à 07:11:00, sortie vide (NULL) : le lot tourne encore.
```

4.4 Lire les lots par profil. Un profil vise une couleur, du plus clair au plus foncé, et le dictionnaire des sources, section 4, donne la perte de poids que chacun vise. La perte se calcule sur les sommes : 1, moins le poids torréfié sur le poids vert.

```
-- 1. par profil : les lots, la phase de développement en moyenne, la perte de poids en pour cent
select profil, count(*) as lots,
       round(avg(secondes_de_developpement)) as developpement,
       round(100 * (1 - sum(____) / sum(____)), 1) as perte
```


© 2026 iAlexMG. Tous droits réservés.

```
from brulerie.fct_lots
group by profil
order by developpement;
-- Attendu : CLAIRE 3 lots, 87 secondes, 13.1 ; MOYENNE 26, 133, 14.9 ; MOYENNE_FONCEE 18, 198, 16.0.
```

Plus le profil est foncé, plus la phase de développement dure, et plus le café perd de poids : 13,1 %, 14,9 % et 16,0 %, au plus près des 13,5 %, 15,0 % et 16,0 % que vise le dictionnaire. Aucun lot du mois n'est torréfié foncé.

4.5 Compter les décisions par statut et par canal. Le fait n'a aucune mesure : `count(*)` est la sienne.

```
-- 1. les décisions, par statut et par canal : le compte des lignes est la mesure
select statut, canal, count(*) as decisions
from brulerie.fct_consentements
group by ____, ____
order by statut;
-- Attendu : ACCORDE CAISSE 547, REFUSE CAISSE 447, RETIRE LIEN 60 ; 1054 décisions en tout.
```

Les 994 membres ont décidé à l'inscription, 547 en acceptant, 447 en refusant ; 60 se sont désabonnés depuis, par le lien d'une infolettre. Aucune autre table ne le sait : la fiche ne garde que le dernier mot.

4.6 Lire la dernière décision de chaque membre. C'est son état d'aujourd'hui, et elle doit tomber sur sa fiche. Elle se tire comme l'état courant du cours 03, par `row_number()` et `qualify`, cette fois par membre, de la plus récente à la plus ancienne.

```
with derniere as (
    -- 1. de chaque membre, sa dernière décision
    select cle_membre, statut
    from brulerie.fct_consentements
    qualify row_number() over (partition by ____ order by horodatage desc) = 1
)
-- 2. les membres, par leur dernière décision
select statut, count(*) as membres
from derniere
group by statut
order by statut;
-- Attendu : ACCORDE 487, REFUSE 447, RETIRE 60 : 994 membres, dont 487 qui consentent.
```

Autant que les 487 membres qui consentent dans `stg_membres`, au cours 03. Deux exports de la plateforme, le journal et la fiche, disent la même chose : c'est le témoin de ce fait, que l'étape 5 rend permanent, membre par membre.

4.7 Fermer le notebook, pour que dbt puisse écrire dans l'entrepôt : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

© 2026 iAlexMG. Tous droits réservés.

Étape 5 — Les tests des deux faits · MARTS

Le cours 05 a posé les règles ; elles s'appliquent ici telles quelles. Le grain d'un fait d'accumulation est le dossier : `lot_id` y est unique et toujours rempli. Chaque clé recalculée existe dans sa dimension. Le statut d'une décision est une liste fermée, avec son `not_null` à côté. Le même fichier décrit les deux faits, et la nouvelle période de `dim_date`.

5.1 Compléter le fichier des `MARTS`, dans la première fenêtre.

```
notepad 02_ae\models\marts\_marts.yml
```

En tête du fichier, deux lignes changent : la première, qui dit maintenant ce que porte le fichier, et la description de `dim_date`, sa nouvelle période.

```yaml
# 02_ae/models/marts/_marts.yml : les dimensions et les trois faits, décrits et testés
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les jours du 1er mai 2023 au 30 juin 2025, un par ligne, fabriqués sans source."
```

Puis, au bout du fichier, les deux faits, alignés sur `fct_ventes`. Les trois codes du statut sont dans le dictionnaire des sources, section 3, « `statut` d'un consentement ».

```yaml
  # 4. le fait d'accumulation : une ligne par lot, et chaque clé dans sa dimension
  - name: fct_lots
    description: "Le fait d'accumulation des lots du torréfacteur, une ligne par lot, un jalon par colonne."
    columns:
      - name: lot_id
        description: "Le numéro du lot : le grain du fait."
        data_tests:
          - ____
          - ____
      # puis, à écrire : cle_date et cle_produit, chacune avec not_null et relationships, comme dans fct_ventes
  # 5. le fait sans mesure : chaque clé dans sa dimension, et le statut dans une liste fermée
  - name: fct_consentements
    description: "Le fait sans mesure des consentements, une ligne par décision d'un membre."
    columns:
      # cle_date et cle_membre, chacune avec not_null et relationships vers la clé de sa dimension
      - name: statut
        description: "Ce que dit la décision : accordé, refusé ou retiré."
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: [____]
```


© 2026 iAlexMG. Tous droits réservés.

Enregistrer, puis fermer le Bloc-notes.

5.2 Écrire le témoin. Il compare, membre par membre, la dernière décision du fait au consentement de la fiche, comme `assert_temoin_caisse` compare chaque ticket. Le fait ne porte que la clé du membre : du côté de la fiche, elle se recalcule par le même `md5`.

```
notepad 02_ae\tests\assert_temoin_consentements.sql
```

Accepter la création, puis écrire :

```
-- 02_ae/tests/assert_temoin_consentements.sql : la dernière décision de chaque membre, telle que sa fiche la porte
with journal as (
    -- 1. de chaque membre, la dernière décision du fait : accordée ou non
    select cle_membre, statut = '____' as consent
    from {{ ref('fct_consentements') }}
    qualify row_number() over (partition by cle_membre order by horodatage desc) = 1
),
fiche as (
    -- 2. le consentement que porte la fiche de chaque membre, sa clé recalculée
    select ____ as cle_membre, consent_infolettre as consent
    from {{ ref('stg_membres') }}
)
-- 3. un membre d'un seul côté, ou deux réponses qui diffèrent : chaque ligne rendue est un échec
select coalesce(j.cle_membre, f.cle_membre) as cle_membre, j.consent as journal, f.consent as fiche
from journal as j
full outer join fiche as f on f.cle_membre = j.cle_membre
where j.cle_membre is null or ____ or j.consent <> f.consent
```

Enregistrer, puis fermer le Bloc-notes.

5.3 Tout lancer.

```
dbt build
REM Attendu : 43 of 43 : treize modèles, le snapshot et 29 tests, puis Done. PASS=43 WARN=0 ERROR=0.
```

Les deux faits ont pris 12 tests génériques et le témoin. `dbt build` les passe chacun juste après son modèle, et avant tout ce qui le lit.

Si dbt s'arrête sur	**La cause, et le geste**
`FAIL 1017 relationships_fct_consentements_cle_date__date_jour__ref_dim_date_`	`dim_date` couvre encore juin seulement : reprendre l'étape 3.1, puis relancer `dbt build`

© 2026 iAlexMG. Tous droits réservés.

Si dbt s’arrête sur	La cause, et le geste
`IO Error: Cannot open file "…\brulerie.duckdb"`	le notebook tient l’entrepôt : quitter DuckDB dans la seconde fenêtre, puis relancer dbt

Étape 6 — Versionner · MARTS

6.1 Valider les deux vues, les deux faits, la dimension des dates étendue et le témoin.

```
git -C 02_ae add models tests
git -C 02_ae commit -m "Cours 06 : les lots et les consentements, deux autres faits"
REM Attendu : 8 files changed.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Un fait d’accumulation suit un dossier : une ligne par dossier, une colonne par jalon, des durées calculées une fois ; sa ligne se remplit à mesure, et un jalon vide dit que l’étape n’a pas eu lieu.
- Un fait sans mesure n’a que des clés et des codes : le compte de ses lignes est sa mesure, et il garde l’histoire que la fiche écrase.
- Une dimension conforme sert plusieurs faits et couvre la période de chacun ; la matrice de bus dit lesquels.

© 2026 iAlexMG. Tous droits réservés.